A PROJECT REPORT ON

Model Compression Toolkit

SUBMITTED TOWARDS THE
PARTIAL FULFILLMENT OF THE ACADEMIC REQUIREMENTS
FOR THE SEMESTER VIII

BACHELOR OF TECHNOLOGY

(Computer Engineering)

Submitted by

Relan Pratiksha Virendra	Exam No: 23054491246054
Joshi Bhagyesh Pradeep	Exam No: 23054491245053
Vora Avinash Ritesh	Exam No: 24054491245504
Joshi Yesha Pratik	Exam No: 24054491245512

Under the Guidance of

Dr. Kavita Patil

SHRI VILE PARLE KELAVANI MANDAL'S
INSTITUTE OF TECHNOLOGY, DHULE 424001

Affiliated to

DR. BABASAHEB AMBEDKAR
TECHNOLOGICAL UNIVERSITY, LONERE

Year: 2026–2027

CERTIFICATE

This is to certify that the project entitled

Model Compression Toolkit

Submitted by

Relan Pratiksha Virendra	Exam No: 23054491246054
Joshi Bhagyesh Pradeep	Exam No: 23054491245053
Vora Avinash Ritesh	Exam No: 24054491245504
Joshi Yesha Pratik	Exam No: 24054491245512

is a bonafide work carried out by the students under the supervision of **Dr. Kavita Patil** and it is submitted towards the partial fulfillment of the academic requirements for Semester VIII (Computer Engineering) of Dr. Babasaheb Ambedkar Technological University, Lonere.

Dr. Kavita Patil
Internal Guide
Dept. of Computer Engineering

Dr. Makrand Shahade
Head of Department
Dept. of Computer Engineering

Dr. Nilesh Salunke
Principal
SVKM's Institute of Technology, Dhule

Signature of Internal Examiner

Signature of External Examiner

PROJECT APPROVAL SHEET

A Project Title

Model Compression Toolkit

is successfully completed by

Relan Pratiksha Virendra	Exam No: 23054491246054
Joshi Bhagyesh Pradeep	Exam No: 23054491245053
Vora Avinash Ritesh	Exam No: 24054491245504
Joshi Yesha Pratik	Exam No: 24054491245512

at

DEPARTMENT OF COMPUTER ENGINEERING

SHRI VILE PARLE KELAVANI MANDAL'S
INSTITUTE OF TECHNOLOGY, DHULE

DR. BABASAHEB AMBEDKAR
TECHNOLOGICAL UNIVERSITY, LONERE

ACADEMIC YEAR 2026–2027

Dr. Kavita Patil
Internal Guide
Dept. of Computer Engineering

Dr. Kavita Patil
Project Coordinator
Dept. of Computer Engineering

Acknowledgements

It gives us great pleasure to present the final-year project report on **"Model Compression Toolkit"**.

We would like to thank our internal project guide, **Dr. Kavita Patil**, for the guidance, patience and critical feedback that shaped this work. The repeated insistence that every claim in the project be backed by a measurement is the reason this report separates what was measured from what was only expected.

We are grateful to **Dr. Makrand Shahade**, Head of the Department of Computer Engineering, for his support and suggestions, and to the Principal, **Dr. Nilesh Salunke**, for the infrastructure, encouragement and support provided to us.

We also thank all teaching and non-teaching staff of the Department of Computer Engineering, and our friends, who helped us directly or indirectly in completing this work. We acknowledge the authors of the open-source libraries on which the toolkit is built, namely PyTorch, Torch-Pruning, ONNX and ONNX Runtime, and the free GPU time provided by Kaggle that made the larger experiments possible.

Relan Pratiksha Virendra
Joshi Bhagyesh Pradeep
Vora Avinash Ritesh
Joshi Yesha Pratik
(B.Tech Computer Engineering)

Abstract

Convolutional neural networks reach good accuracy only with parameter counts and arithmetic that assume a datacentre accelerator, while the processors that actually run them in laptops, kiosks and embedded controllers are general-purpose CPUs. Three techniques reduce a trained network for such targets: structured pruning, knowledge distillation and quantization. Published work usually reports their benefit through proxies such as FLOPs and file size, and a smaller file or a lower FLOP count does not guarantee a faster model on a given CPU.

This project builds **CompressKit**, a Python toolkit that composes the three techniques in a fixed, justified order (prune, recover, distill, quantize last) and measures every reported quantity on the exported ONNX file through ONNX Runtime. Four mechanisms distinguish it: an instruction-set probe that permits INT8 only on hosts with fused low-precision kernels; a per-layer SIMD alignment rule that keeps narrow layers from being silently left unpruned; a shared pruning ratio across the layers that residual connections tie together; and a label-free per-layer sensitivity analysis that supplies both the quantization exclusions and non-uniform pruning ratios. A declarative budget search escalates from quantization alone to pruning and distillation only when a stated latency, accuracy or size budget is still unmet. The toolkit produces a compressed `.onnx` file, a detailed HTML report and a machine-readable `results.json`.

The toolkit is about 5,000 lines of Python with 162 automated tests. On CIFAR-10 it compressed ResNet-20 from 1.05 MB to 0.11 MB (9.1 times smaller) and from 1.45 ms to 0.46 ms on an Apple M5, at a cost of 7.54 percentage points of accuracy (91.54% to 84.00%). A VGG-16-BN run on a Kaggle CPU reduced the model 15.7 times at a 3.02-point accuracy cost. The report also records results that did not favour the method: at an equal fine-tuning budget one-shot pruning matched or beat gradual pruning on ResNet-20, and batch-norm re-estimation alone recovered little accuracy. Verification of the compressed ONNX file against the original on real test images is included, together with the limits of what was tested.

Keywords: model compression, structured pruning, knowledge distillation, post-training quantization, ONNX Runtime, SIMD alignment, layer sensitivity, edge inference.

Contents

1 Synopsis **1**
- 1.1 Project Title . . . 1
- 1.2 Domain of the Project . . . 1
- 1.3 Problem Statement . . . 1
- 1.4 Objectives and Expected Outcomes . . . 1

2 Introduction **2**
- 2.1 Overview of the Project Domain . . . 2
- 2.2 Project Purpose and Applicability . . . 2
- 2.3 Scope of the Project . . . 3
- 2.4 Organisation of the Report . . . 3

3 Literature Survey **4**
- 3.1 Related Work . . . 4
- 3.2 Summary of the Surveyed Papers . . . 5
- 3.3 Limitations of Existing Systems . . . 5
- 3.4 Motivation for the Proposed System . . . 5

4 Project Implementation Plan **6**
- 4.1 Project Schedule . . . 6

5 Project Requirement Specification **7**
- 5.1 Hardware and Software Requirements . . . 7
- 5.2 Functional Requirements . . . 7
- 5.3 Non-Functional Requirements . . . 8
- 5.4 Dataset Description . . . 8

6 Proposed System **9**
- 6.1 Overall System Architecture . . . 9
- 6.2 Layered Architecture and Modules . . . 9
- 6.3 The Compression Pipeline in Detail . . . 11
- 6.4 Core Algorithms . . . 13
- 6.5 Data Flow Diagrams . . . 16
- 6.6 UML Diagrams . . . 18

7 System Implementation **20**
- 7.1 Package Layout . . . 20
- 7.2 Command-Line Interface . . . 20
- 7.3 Selected Code . . . 20
- 7.4 One-Notebook Workflow . . . 21
- 7.5 A Defect Found by Verification . . . 21

8 Results and Testing **22**
- 8.1 Experimental Setup . . . 22
- 8.2 Compression Results . . . 22

8.3 Channels Removed . . . 24
8.4 Gradual Pruning . . . 24
8.5 Batch-Norm Re-estimation . . . 26
8.6 Distillation Signal . . . 26
8.7 Quantization . . . 27
8.8 Layer Sensitivity . . . 27
8.9 Budget Search . . . 28
8.10 Verification of the Compressed ONNX File . . . 28
8.11 Testing . . . 29
8.12 Limitations . . . 29
9 Conclusion and Future Scope **30**
9.1 Summary . . . 30
9.2 Conclusions . . . 30
9.3 Future Scope . . . 30
Bibliography **31**

List of Figures

2.1 Order of operations in CompressKit. Quantization is applied last because integer weights cannot be pruned or trained further. . . . 2

4.1 Project timeline from commit dates and file modification dates. The hatched band is an axis break (17 September to 10 October, no recorded work). Dots show commits per day. . . . 6

6.1 Overall system architecture of CompressKit: inputs, the toolkit with its stages, search, measurement and reporting, and the three outputs. . . . 9
6.2 Layered architecture of CompressKit. Directory names match the source tree. 10
6.3 The pipeline in detail: the six stages, the pruning loop and the quantization steps expanded, and the measurement performed after every stage. . . . 11
6.4 Channels kept against requested ratio for a layer of width 16 (left) and 64 (right) under alignments 8, 16 and 32, computed by the toolkit's `_aligned_ratio`. A narrow layer cannot satisfy a large alignment, so all curves coincide on the left. . . . 14
6.5 Escalation ladder of the budget search. Colours distinguish the rungs; the cheapest is on the left. . . . 16
6.6 Data flow diagram, level 0: the toolkit as one process between the user, the dataset and the target runtime. . . . 16
6.7 Data flow diagram, level 1: the six processes of one compression run. . . . 17
6.8 Data flow diagram, level 2: the pruning process. . . . 17
6.9 Use case diagram. The target host CPU is the secondary actor because latency and INT8 support are properties of that machine. . . . 18
6.10 Sequence of a budget-driven `compress` run. . . . 18
6.11 Class diagram of the principal types. . . . 19

8.1 Accuracy, size and median latency through the four stages for the three recorded runs (size and latency on log axes). . . . 22
8.2 Output channels per layer of ResNet-20 before and after pruning. . . . 24
8.3 Accuracy after each cut and after each fine-tune. ResNet-20 on Apple M5; VGG-16-BN transcribed from the Kaggle run. . . . 25
8.4 Gradual and one-shot pruning at the same fine-tune budget on ResNet-20, one seed each. . . . 25
8.5 Running variance of one batch-norm layer before and after the cut (left) and the accuracy effect (right). . . . 26
8.6 Teacher and student softmax on one test image at temperature 1 (left) and 4 (right). . . . 26
8.7 Weight histogram of one layer before and after INT8 (left) and the INT8 scale of each output channel (right). . . . 27
8.8 Per-layer SQNR of ResNet-20 after INT8 round trip of one layer at a time. Lower means more sensitive. . . . 27
8.9 Every attempt made under each budget. Green bars met the budget; the dashed line is the baseline accuracy. . . . 28

List of Tables

3.1 Ten surveyed papers and the gap each leaves open. . . . 5

5.1 Hardware used. The probe results are those recorded by the toolkit's host module. . . . 7
5.2 Software stack. . . . 7

6.1 Pipeline stages with inputs and guards. . . . 12

8.1 Experimental runs and their status. . . . 22
8.2 Per-stage measurements. ResNet-20 and ResNet-56 on Apple M5; ResNet-56 is the short smoke schedule; VGG-16-BN on a Kaggle CPU, transcribed from the run log. . . . 23
8.3 Compression ratios, each beside the accuracy it cost. The VGG-16-BN speed-up is on a CPU without fast INT8 and is not a deployment figure. . . . 23
8.4 Outcomes of the budget search. The third budget is infeasible by design. . . . 28

1. Synopsis

1.1 Project Title

Model Compression Toolkit (implemented as the Python package *CompressKit*): a latency-verified compression toolkit for convolutional neural networks.

1.2 Domain of the Project

Efficient deep learning: structured pruning, knowledge distillation and post-training quantization, with performance measurement on CPU inference runtimes.

1.3 Problem Statement

A trained CNN is usually too large and too slow for the CPU it must run on. Compression with pruning, distillation or quantization raises three recurring problems. First, the techniques are published as separate scripts, so their order and interaction are left to the user, although the order matters: quantization cannot be undone and blocks further training. Second, results are reported as FLOP counts or file sizes, which can disagree with measured latency; INT8 on a CPU without fused integer kernels can shrink the file and still run slower than FP32. Third, a user with a latency or accuracy requirement has no tool that chooses how much compression is needed and reports honestly when the requirement cannot be met.

The problem addressed here is to build a toolkit that, given a trained CNN and optionally a stated budget, produces a compressed ONNX model with measurements taken on the exported file, and states where its guarantees stop.

1.4 Objectives and Expected Outcomes

- Implement structured channel pruning that handles residual connections and keeps channel counts SIMD-aligned.
- Implement gradual pruning with interleaved fine-tuning, batch-normalisation re-estimation, and KD and decoupled KD recovery.
- Implement INT8 and FP16 export through ONNX Runtime, quantization-aware training as a fallback, and a per-layer sensitivity analysis.
- Implement a budget-driven search that applies the cheapest sufficient combination of the above.
- Measure accuracy, latency and size on the exported ONNX file under one protocol, and generate an HTML report.
- Provide one reproducible workflow (one notebook and a command-line interface) that runs end to end on a free GPU service.
- Demonstrate the toolkit on more than one network and verify the compressed file against the original on real test images.

2. Introduction

2.1 Overview of the Project Domain

Convolutional neural networks (CNNs) are the standard solution for image classification, and their accuracy comes with cost: ResNet and VGG families carry millions to hundreds of millions of parameters and tens to thousands of millions of multiply-accumulate operations per image [14, 15]. Deploying them on CPUs requires reducing the memory footprint of the weights and the time of one forward pass.

Three technique families dominate the literature.

Pruning removes structure from the network. Unstructured pruning zeroes individual weights, which leaves every tensor the same size and gives no speed-up on a standard CPU runtime. Structured pruning removes whole channels or filters, which shrinks the matrix multiplications themselves. This project uses structured pruning only.

Quantization lowers the numeric precision of weights and activations, typically from 32-bit float to 8-bit integer (INT8) or 16-bit float (FP16) [17]. File size falls in proportion to the bit width, but latency falls only when the processor has instructions that compute directly in the lower precision.

Knowledge distillation trains a smaller student against the soft output distribution of a larger teacher [12]. Decoupled KD splits the loss into a target-class and a non-target-class term that can be weighted separately [13].

Fig. 2.1 shows the order in which CompressKit applies them.



Figure 2.1: Order of operations in CompressKit. Quantization is applied last because integer weights cannot be pruned or trained further.

2.2 Project Purpose and Applicability

The purpose is to make compression a repeatable engineering step with a measurable result instead of a sequence of ad-hoc scripts. The toolkit applies to CNN image classifiers that export to ONNX, on the CPU targets where ONNX Runtime's CPU provider is used: laptops, kiosks, industrial controllers and single-board computers. Typical use

is to take a model trained for a specific task, state a budget such as "at most 0.45 ms median latency and at most 8 points of accuracy loss", and obtain a file that meets it or a statement that it cannot.

2.3 Scope of the Project

In scope. Classification CNNs with convolution, batch normalisation, residual or plain stacking and a linear classifier head; structured pruning, KD and DKD, INT8 and FP16 post-training quantization, QAT as a fallback; CIFAR-10, a human-action-recognition image dataset, and any image-folder dataset; CPU measurement through ONNX Runtime; a single-notebook workflow on Kaggle and a command-line interface.

Out of scope. Object detection and segmentation models; transformer architectures; GPU or NPU latency; training from scratch beyond the baselines needed to have something to compress. Depthwise-separable networks (MobileNet style) were not exercised and are not claimed.

2.4 Organisation of the Report

Chapter 3 surveys ten related papers and extracts the gaps that motivate the design. Chapter 4 gives the project plan and timeline. Chapter 5 specifies the requirements and datasets. Chapter 6 presents the proposed system: architecture, algorithms and diagrams. Chapter 7 describes the implementation. Chapter 8 reports measured results, verification and the testing. Chapter 9 concludes and lists future work.

3. Literature Survey

3.1 Related Work

Ten papers were studied in detail, spanning pipeline integration, structured pruning, quantization and distillation. For each, the reported claims were checked against the paper's own tables; discrepancies are noted because they shape what the toolkit must verify rather than inherit.

Pipeline integration. Chae and Seo [1] argue that quantization must be applied last and propose prune, distillation-assisted fine-tuning, quantization-aware training and INT8 export, reusing the uncompressed model as the teacher at two points (compression-aware KD). Results on ResNet-56 and MobileNetV2 are reported in FLOPs and size; GPU hardware is used only for training, and two FLOP tables for the same configuration are inconsistent. Wang et al. [2] alternate transfer KD and adaptive channel pruning every epoch rather than pruning once. Their headline gains of up to 18.6 points are best-single-task comparisons against the weakest baseline; the aggregate gain over the strongest baseline is 4.4 points, and no latency is measured.

Structured pruning. He et al.'s AMC [3] learns per-layer pruning ratios with a reinforcement-learning agent and reports measured speed-ups of 1.53 times on GPU and 1.95 times on a phone, one of the few papers in the set with on-device latency. The abstract's 2.7-point gain does not reconcile with the paper's own VGG-16 table, where the margin is 0.3 points. Zhou et al.'s AEMC [4] identifies an amplitude mismatch when blocks of a residual network are removed and corrects it with a parameter-free rescaling; the aggregate gain hides a per-task collapse on one benchmark, and no latency is reported.

Quantization. Chen et al. [5] study accuracy against bit width and observe memory reductions of 3.5 to 6.4 times but no speed-up on unmodified CPU or GPU, a direct warning that INT8 file savings do not imply latency savings. VLCQ [6] adds variable-length coding to data-free quantization and reaches 12.26 times effective compression at a 3.89-point loss in one best-case configuration, with decoder throughput asserted rather than measured. Khan and Rafat [7] combine filter pruning with K-means quantization and report 34,052 times compression, but the abstract's accuracies are training accuracies and the test accuracies are 87.39% and 53.61%, and the distillation loss described is not the one used in the experiments. Fang et al. [8] add fake quantization with a straight-through estimator to federated learning; the convergence bounds for the proposed and baseline methods are algebraically identical.

Distillation. Chen et al. [9] combine feature and noisy-logit distillation with iterative magnitude pruning; the implied compression is 48.4 times rather than the stated 49.5 times, and the claimed speed-up on a Raspberry Pi cannot be attributed to it because the uncompressed teacher did not fit in memory there. Xu et al.'s ReKD [10] splits logits by rank into head and tail categories and improves on classical KD by 0.6 to 3.7 points, while claiming no extra hyperparameters despite introducing three and reporting no runtime.

Ref.	Topic	Main idea	Gap found on closer reading
[1]	Pipeline	Prune, KD, QAT, INT8; original model as teacher	FLOPs and size only; inconsistent FLOP tables
[2]	Pipeline	Alternate KD and channel pruning each epoch	Best-case gains; no latency
[3]	Pruning	RL-chosen per-layer ratios	Abstract gain not in own table
[4]	Pruning	Rescale after removing residual blocks	Per-task collapse; no latency
[5]	Quantization	Accuracy against bit width	No speed-up without matching hardware
[6]	Quantization	Variable-length coded weights	Best-case ratio; decoder speed unmeasured
[7]	Quantization	Pruning plus K-means	Train accuracy in abstract
[8]	Quantization	QAT with STE in federated learning	Identical bounds
[9]	Distillation	Noisy-logit KD plus pruning	Ratio and Raspberry Pi claims
[10]	Distillation	Head and tail category KD	Hidden hyperparameters; no runtime

Table 3.1: Ten surveyed papers and the gap each leaves open.

3.2 Summary of the Surveyed Papers

3.3 Limitations of Existing Systems

Three patterns recur across the set. *Proxy substitution*: FLOPs and parameter counts are reported far more consistently than measured on-device latency, and in five of the ten papers hardware mentioned in the text is used only for training. *Best-case reporting*: abstracts quote the best single configuration or the weakest baseline. *Train and test conflation*: the extreme case is [7], where the quoted accuracy is the training accuracy.

On the tool side, the common open implementations provide the individual techniques (for example Torch-Pruning for structured pruning [11] and ONNX Runtime's static quantizer), but not the ordering, the hardware gating, the layer-aware allocation, or the budget search that ties them to a requirement.

3.4 Motivation for the Proposed System

The gaps translate into three evaluation rules that the toolkit enforces: benchmark on the deployment runtime instead of reporting proxies, report what was measured (including the cases where the method loses), and validate every stage on a held-out set rather than on training data. A fourth motivation is practical: a user who has only a latency and accuracy requirement should not have to choose and order the techniques by hand.

4. Project Implementation Plan

4.1 Project Schedule

4.1.1 Project Task Set

The major tasks of the project were:

- **Task 1** Study of compression literature and definition of the evaluation rules (Chapter 3).
- **Task 2** Core infrastructure: host probe, ONNX export, benchmarking protocol.
- **Task 3** Compression stages: structured pruning, BN recalibration, KD and DKD, quantization, QAT, sensitivity analysis.
- **Task 4** Orchestration: the staged pipeline, the budget search, the HTML report.
- **Task 5** Hardening through edge-case tests and review of silent-failure modes.
- **Task 6** Gradual pruning, dataset-agnostic CLI, and the single-notebook Kaggle workflow.
- **Task 7** Experiments on ResNet-20, ResNet-56 and VGG-16-BN; verification of the ONNX output; documentation and this report.

4.1.2 Time Line Chart

Fig. 4.1 lays the work out against dates that can be checked in the repository: commit dates (41 commits on six days) and the modification dates of the collected papers, the survey and design documents, the training logs and the draft paper. Bars therefore start with the literature work on 5 September, run through the package split and hardening in mid-September, and continue with the budget-search work on 11 October and the gradual-pruning and Kaggle workflow on 26 and 27 October. No work is recorded between 17 September and 10 October, so that stretch is shown as a hatched break instead of being filled in.

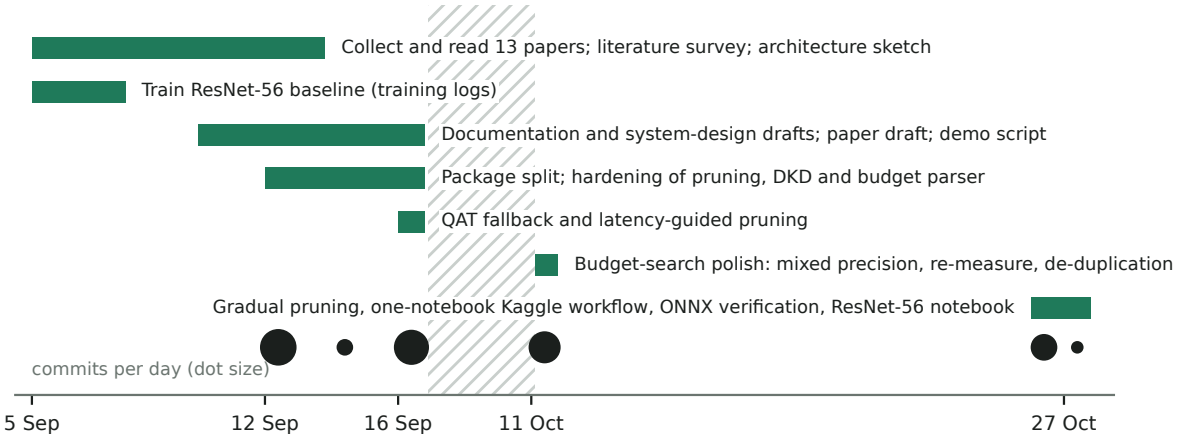


Figure 4.1: Project timeline from commit dates and file modification dates. The hatched band is an axis break (17 September to 10 October, no recorded work). Dots show commits per day.

5. Project Requirement Specification

5.1 Hardware and Software Requirements

5.1.1 Hardware

Two machines were used, with different roles. The Apple laptop is the deployment-like target on which latency is judged; the Kaggle machine supplies a GPU for training and fine-tuning.

Sr.	Parameter	Local machine (final timing)	Kaggle notebook (training)
1	Processor	Apple M5, 10 logical cores (4 performance)	Intel Xeon @ 2.00 GHz, 4 logical cores
2	INT8 support	Fast: i8mm, dotprod, fp16, bf16, neon	Slow: avx2, avx512f, f16c only
3	Accelerator	None used	NVIDIA T4 GPU
4	Role	Latency and ONNX verification	Training, fine-tuning, compression

Table 5.1: Hardware used. The probe results are those recorded by the toolkit's host module.

5.1.2 Software

Component	Version / choice	Purpose
Python	3.14.7 (local)	Implementation language
PyTorch	2.14.0	Model definition, training, fine-tuning
Torch-Pruning	DepGraph pruner	Dependency-aware channel removal
ONNX Runtime	1.29.0	Quantization and all measurements
NumPy	2.5.2	Array operations, CPU feature flags
pytest	configured in `pyproject.toml`	162 tests by default
Git and GitHub	–	Version control and distribution
Kaggle Notebook	T4 GPU, internet on	Single end-to-end run environment

Table 5.2: Software stack.

5.2 Functional Requirements

1. FR1: Train or load an FP32 classification baseline for CIFAR-10, a human-action-recognition dataset, or any one-folder-per-class image dataset.

2. FR2: Prune a model structurally by a requested ratio, in one or several steps, with per-layer alignment and shared ratios across residual groups.

3. FR3: Re-estimate batch-norm statistics and fine-tune with KD or DKD against the original model as teacher.

4. FR4: Quantize to INT8 or FP16 through ONNX Runtime, with an automatic choice gated by the host, and fall back to QAT when asked.

5. FR5: Rank layers by quantization sensitivity and use the ranking for exclusions and pruning ratios.

6. FR6: Accept a budget (latency, accuracy drop, minimum accuracy, size) and search for the cheapest configuration meeting it.

7. FR7: Benchmark every stage on the exported ONNX file and write `results.json`, an HTML report and the `.onnx` file.

8. FR8: Verify the compressed file against the original on real test images.

5.3 Non-Functional Requirements

- **Measurement integrity.** Latency uses one ONNX Runtime session configuration for every precision: one thread, batch size 1, 50 warm-up runs, 200 timed runs, median and 95th percentile reported. Accuracy is measured on the exported file.

- **No leakage.** BN re-estimation and quantization calibration read only the training split.

- **Reproducibility.** The train/test split of folder datasets is deterministic by sorted file name; random seeds are fixed for calibration subsets.

- **Honest failure.** Unmet budgets return the best trade-off and say so. Operators without an INT8 kernel are counted in the report.

- **Usability.** One command or one notebook run produces all outputs.

- **Portability.** Pure Python, CPU inference, no vendor-specific runtime.

5.4 Dataset Description

CIFAR-10 [16] 60,000 colour images of 32×32 pixels in 10 classes: 50,000 for training and 10,000 for testing. Used for ResNet-20, ResNet-56 and VGG-16-BN. Input shape for the toolkit: $1 \times 3 \times 32 \times 32$.

Human Action Recognition (HAR) A 15-class image dataset at 224×224 input, read through a loader that finds the data under the Kaggle input directory. The ResNet-18 baseline path is implemented; its Kaggle run was not completed within this project, so no HAR accuracy is claimed.

Image folder Any dataset with one sub-directory per class. A fixed 15% of each class, taken from the end of the sorted file list, is held out as the test set.

6. Proposed System

6.1 Overall System Architecture

Fig. 6.1 shows the whole system from what the user supplies to what it delivers. Three inputs go in: a trained FP32 checkpoint, a dataset with train, test and calibration views, and a request, which is either a fixed recipe (prune ratio, steps, fine-tune epochs, precision) or a budget string. The toolkit itself has five cooperating parts. An *entry point* (command line, Python API or notebook) builds a `Pipeline`. The *host probe* reads the CPU's instruction-set features and decides whether INT8 is allowed. The *pipeline controller* checks the stage order and runs a pre-flight summary. The *stages* transform the model in a fixed order. The *search layer* sits beside them and, for a budget request, decides which stages to use. After every stage the *measurement* component exports the model to ONNX and measures it in ONNX Runtime, and the *reporting* component turns all measurements into the three outputs: the compressed `.onnx`, an HTML report and `results.json`. Finally the delivered file is re-measured on the target host and compared with the original.

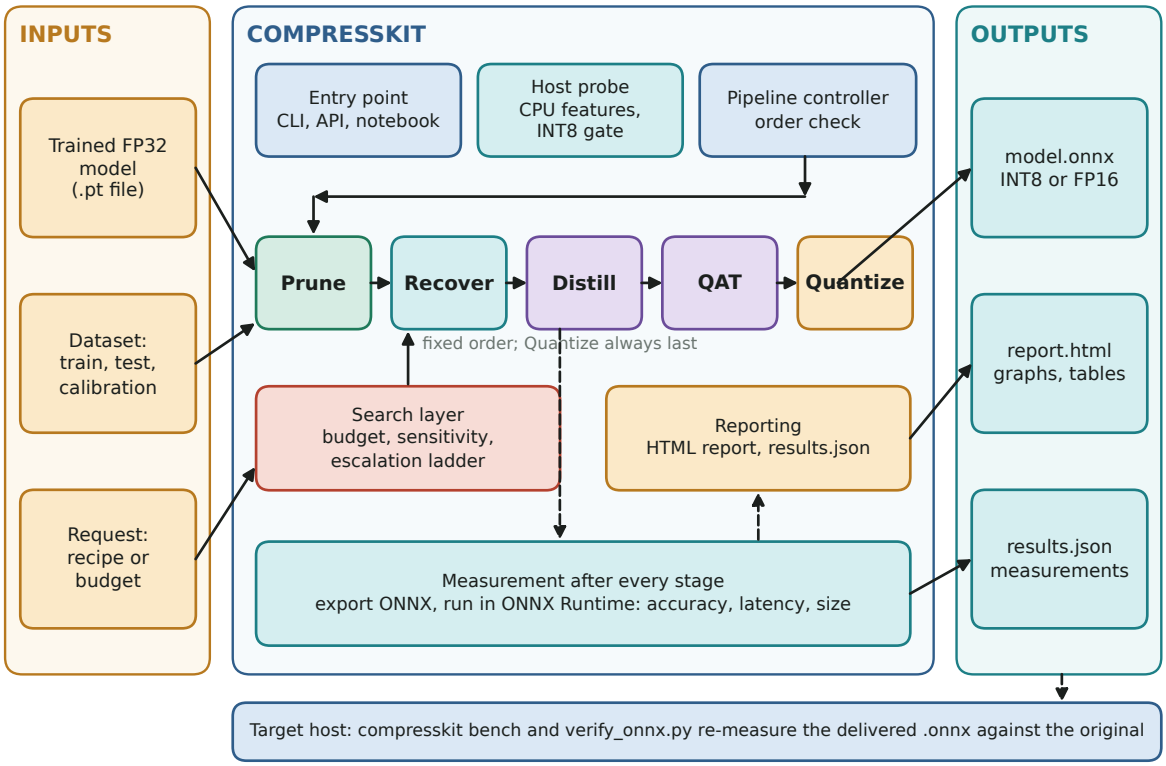


Figure 6.1: Overall system architecture of CompressKit: inputs, the toolkit with its stages, search, measurement and reporting, and the three outputs.

6.2 Layered Architecture and Modules

Seen as a code base, CompressKit is a small library with three entry points, four compression stages, an orchestration layer and a shared core (Fig. 6.2). The core is cross-

cutting: the host probe attaches the machine's capabilities to every measurement, the benchmark module produces the numbers all stages are judged by, and the export module writes ONNX.

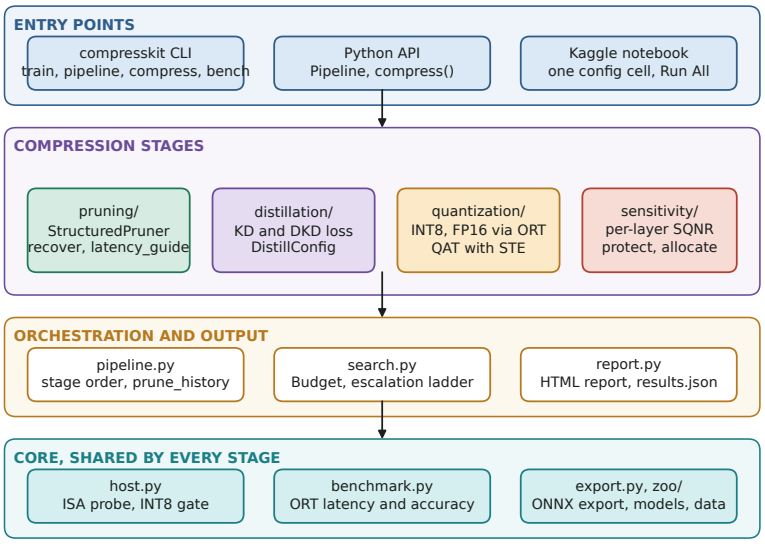


Figure 6.2: Layered architecture of CompressKit. Directory names match the source tree.

6.2.1 Modules

Host probe (`core/host.py`). Reads the CPU's instruction-set features. INT8 is recommended only when at least one of AMX-INT8, AVX-512 VNNI, AVX-VNNI, I8MM or DotProd is present, because without a fused integer dot-product INT8 is routinely slower than FP32 while still producing a smaller file. Otherwise FP16 is the default and INT8 becomes an explicit, warned opt-in. The probe result is stored with each measurement.

Structured pruner (`pruning/prune.py`). Wraps Torch-Pruning's dependency graph [11]. It never prunes the classifier, always works on a deep copy of the model, and adds three behaviours described next: per-layer SIMD alignment, shared ratios across residual groups, and gradual steps.

Recovery (`pruning/recover.py`). Resets batch-norm running statistics and recomputes them as a cumulative average over training batches, so the stale statistics of the old network are not applied to the new one.

Distillation (`distillation/distill.py`). KD and decoupled KD with the original, uncompressed model as teacher, so no second network has to be trained [1].

Quantization (`quantization/`). Static post-training quantization through ONNX Runtime with per-channel weights, calibrated on a subset of the training split, plus an operator-coverage audit; QAT with a straight-through estimator [18] as a fallback.

Sensitivity (`sensitivity/analysis.py`). Label-free per-layer ranking used for exclusions and non-uniform pruning ratios.

Pipeline, search and report (`pipeline.py`, `search.py`, `report.py`). Stage orchestration, the budget search, and the HTML and JSON output.

6.3 The Compression Pipeline in Detail

6.3.1 Stage Model and Ordering Rule

The pipeline is a chainable builder: `Pipeline(model, calibration, test, train)` followed by calls such as `.prune(0.5, steps=3, fine_tune_epochs=2).recover().quantize()` only *records* the stages. Nothing runs until `run()`. Two copies of the model are made at construction. One is the untouched *original*, which is both the accuracy reference and the distillation teacher, so it has to survive every in-place stage; the other is the working model.

One rule is enforced before any compute is spent: `quantize` must be the last stage, otherwise `run()` raises an error. Integer weights cannot be pruned structurally and cannot be trained further, so any stage placed after quantization would either fail or silently undo it [1]. A `preflight()` call prints the host capabilities, parameter count, calibration size, the protected classifier layer and the planned stage list, and warns if quantize is not last.

6.3.2 Stage by Stage

Fig. 6.3 gives the data path and Table 6.1 summarises what each stage takes, does and records.

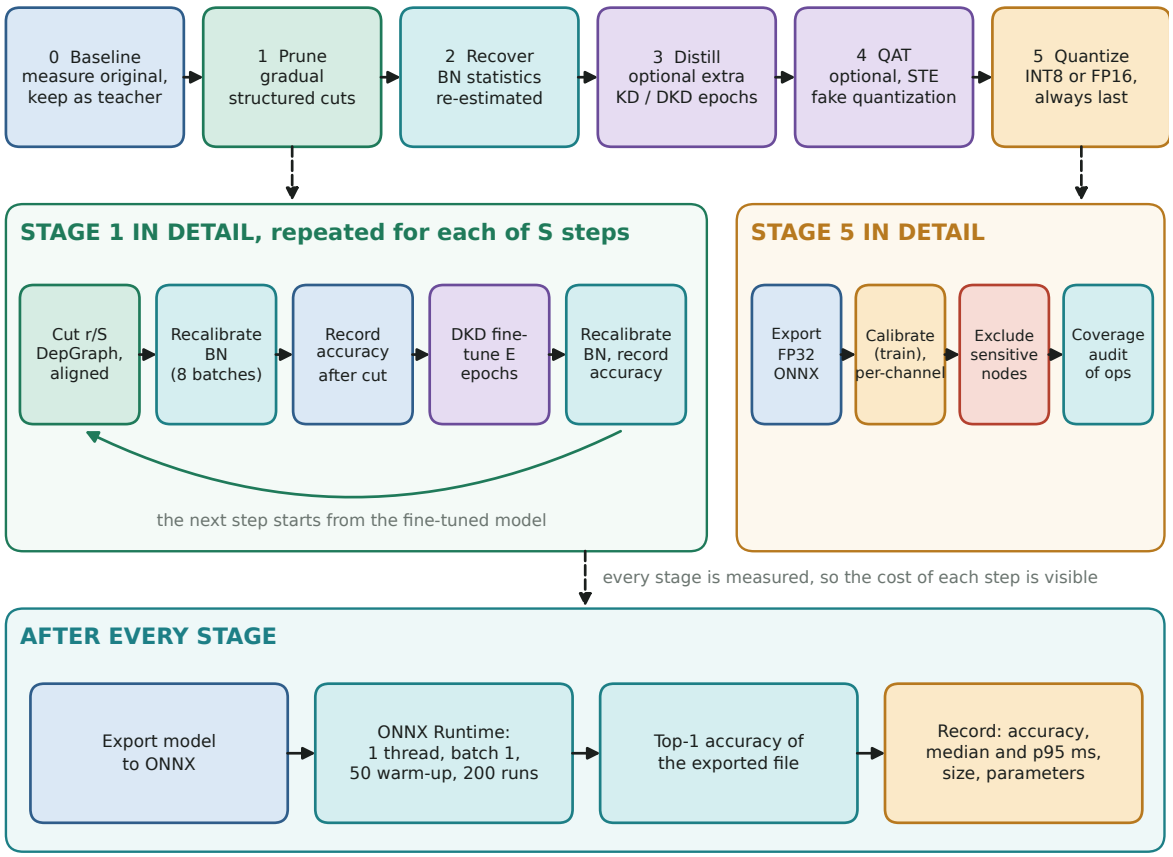


Figure 6.3: The pipeline in detail: the six stages, the pruning loop and the quantization steps expanded, and the measurement performed after every stage.

Pruning loop. Each of the S steps cuts a fraction of the channels, recalibrates the

Stage	What it does	Needs	Guard or record
0 Baseline	Exports the original to ONNX and measures it	Test split	Teacher and reference for all later stages
1 Prune	S cuts of r/S of the channels, each followed by BN recalibration and DKD fine-tuning	Calibration and train loaders	Classifier never pruned; works on a deep copy; `prune_history` rows
2 Recover	Resets BN running statistics and recomputes them as a cumulative average	Calibration loader (train split)	No labels, no back-propagation
3 Distill	Extra KD or DKD epochs against the original	Train loader	Error if no training data is given
4 QAT	Fake quantization with a straight-through estimator	Train loader	Fallback when post-training INT8 misses accuracy
5 Quantize	Exports FP32 ONNX, calibrates, writes INT8 or FP16	Calibration loader, host probe	Sensitive layers excluded; operator coverage audited; must be last

Table 6.1: Pipeline stages with inputs and guards.

batch-norm statistics on eight calibration batches, records the accuracy right after the cut, fine-tunes for E epochs with decoupled KD against the original, recalibrates again and records the accuracy after the fine-tune. Recalibrating between steps matters: the next importance ranking would otherwise be computed on a mis-normalised network. The last cut has no step after it, so it receives the same cut, fine-tune, recalibrate treatment explicitly. This is the code path where the defect described in Section 7.5 was found: the pruner works on its own deep copy, so the final fine-tune must act on the pruner's model and not on the caller's still-unpruned variable.

Recover and distill. Recovery needs no labels and costs a few forward passes. Distillation spends epochs of training and is used when fine-tuning inside the pruning loop is not enough.

Quantize. The working model is exported to an FP32 ONNX file; layer names from the sensitivity ranking are mapped to node names in *that* exported graph; ONNX Runtime's static quantizer calibrates on the training subset with per-channel weight scales; and the result is audited for how many compute operators actually received an INT8 kernel. If the host probe reports no fused integer instructions, the default precision is FP16 and INT8 requires an explicit opt-in.

Measurement. After every stage the model is exported and measured under the same protocol. A stage's cost is therefore visible, and a stage that did not help (such as recovery after fine-tuning, in Chapter 8) shows as a flat step instead of being assumed useful.

6.3.3 The Run Procedure

Algorithm 1: `Pipeline.run`

Input: ordered stages $T_1 \dots T_n$, original model M_0, loaders
Output: list of measurements, prune history, warnings

if *quantize appears and is not last* **then** raise error;
baseline $\leftarrow$ MEASURE(M_0); $M \leftarrow$ copy(M_0);
for $i \leftarrow 1$ **to** n **do**
  switch *kind of* T_i **do**
    case *prune* **do**
      run Algorithm 3 on M; $M \leftarrow$ pruner's model
    case *recover* **do**
      recalibrate BN of M on calibration batches
    case *distill* **do**
      distill M from M_0
    case *qat* **do**
      fake-quantize and fine-tune M
    case *quantize* **do**
      export M; calibrate; write INT8 or FP16; audit coverage
  append MEASURE(result of T_i) to the list;
return measurements, history, warnings;

6.3.4 Why This Order

Pruning first, because it changes the architecture and everything after must fit the smaller network. Batch-norm recalibration and distillation next, because they repair the accuracy lost by the cut while all weights are still float and trainable. Quantization last, because converting weights to integers is hard to reverse and blocks further training. The same order is used by the budget search, which differs only in *how many* of these stages it applies.

6.3.5 Two Ways to Drive the Pipeline

In *fixed-recipe mode* the user states the stages and their parameters, for example `compresskit pipeline --prune 0.5 --steps 3 --fine-tune-epochs 2`. In *budget mode* the search layer builds one pipeline per ladder rung (Section 6.4.5), runs it, compares the measurements with the budget and escalates only if the budget is unmet. Both modes call the same `Pipeline.run`, so every number in a report was produced by the same code path.

6.4 Core Algorithms

6.4.1 Pruning: Residual Groups and Alignment

In a residual network the output channels of a convolution may be summed with a shortcut, so several layers must keep exactly the same channel indices. Torch-Pruning exposes these as dependency groups. In ResNet-20, 12 groups tie two or more layers:

three stage-wide groups of eight layers each, and nine groups that pair a convolution with its batch norm. If two layers in one group request different ratios the pruner would pick one arbitrarily. CompressKit resolves each group toward its most protective member, that is, the lowest requested ratio.

Channel counts are rounded to multiples of a SIMD-friendly alignment (8, 16 or 32). A single global alignment fails for narrow layers: Torch-Pruning floors the kept count to a multiple of the alignment, a layer whose width is under two alignment blocks floors to zero, the group is rejected as invalid, and the layer silently keeps every channel. The toolkit instead computes the nearest achievable aligned count per layer and halves the alignment only for the layer that needs it (Algorithm 2, Fig. 6.4).

Algorithm 2: Per-layer aligned ratio (from `_aligned_ratio`)

Input: layer width w, requested ratio r, alignment a
Output: effective ratio r', alignment used a'

$a' \leftarrow a$;
while $a' > 1$ ***and*** $\lfloor w/a' \rfloor < 2$ **do**
  $a' \leftarrow \lfloor a'/2 \rfloor$
if $a' \leq 1$ **then**
  $k \leftarrow \max(1, \mathrm{round}(w(1-r)))$; **return** $(1 - k/w,\ a')$
$n \leftarrow \min\left(\lfloor w/a' \rfloor,\ \max(1,\ \mathrm{round}(w(1-r)/a'))\right)$;
$k \leftarrow n\,a'$; **return** $(1 - k/w,\ a')$;

For example, a 64-channel layer at requested ratio 0.7 keeps 16 channels with alignment 16 (effective ratio 0.75) and 32 channels with alignment 32 (effective ratio 0.5): the result is the nearest multiple, not the floor. A 16-channel layer at ratio 0.5 keeps 8, and an 8-channel layer keeps 4 by relaxing the alignment to 4. These values come from a 180-case table generated by the real function, which is kept as a golden test of the alignment rule.

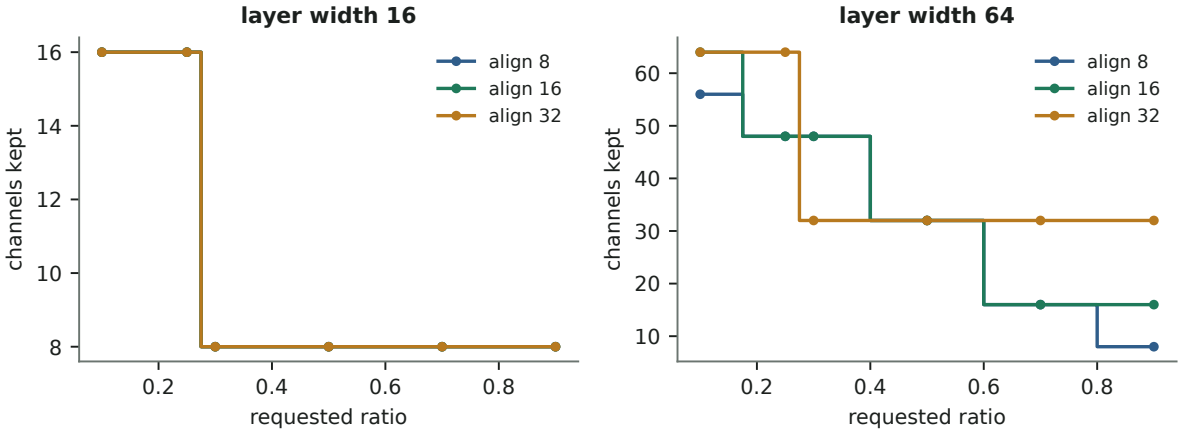


Figure 6.4: Channels kept against requested ratio for a layer of width 16 (left) and 64 (right) under alignments 8, 16 and 32, computed by the toolkit's `_aligned_ratio`. A narrow layer cannot satisfy a large alignment, so all curves coincide on the left.

6.4.2 Gradual Pruning with Fine-Tuning

Instead of removing all channels at once, the pruner removes them in S equal steps and fine-tunes after each (Algorithm 3). The motivation is that a smaller cut is easier to

recover from [2]; whether this beats one-shot pruning at an equal fine-tuning budget is tested in Chapter 8. Every step is recorded as the accuracy right after the cut and right after the fine-tune.

Algorithm 3: Gradual structured pruning with interleaved distillation

Input: trained model M_0 (teacher), ratio r, steps S, epochs per step E
Output: pruned model M_S, history of (acc. after cut, acc. after fine-tune)

$P \leftarrow \textsc{StructuredPruner}(\text{copy}(M_0), r, S)$;
for $s \leftarrow 1$ **to** S **do**
  $M \leftarrow P.\text{step}()$; `// remove` r/S `of the way`
  record accuracy of M; recalibrate BN statistics on train batches;
  if $E > 0$ **then**
    fine-tune M for E epochs with DKD against M_0; record accuracy
return M (the pruner's own model, not the original);

6.4.3 Distillation Loss

With student logits z_s, teacher logits z_t, temperature T and target class c, decoupled KD [13] is

$$\mathcal{L}_{\text{DKD}} = \alpha\,\text{TCKD} + \beta\,\text{NCKD}, \tag{6.1}$$

where TCKD is the KL divergence between the two-way distributions $(p_c,\ 1 - p_c)$ of teacher and student, and NCKD is the KL divergence between the teacher and student distributions over the non-target classes after masking the target before the softmax. Both terms are scaled by T^2. Defaults are $T = 4$, $\alpha = 1$ and $\beta = 8$; plain KD uses $\alpha = 0.3$. The final loss adds the cross-entropy on the labels. The target probability is clamped away from 0 and 1 so that a confident student cannot produce $\log 0$ and a NaN loss.

6.4.4 Quantization and Sensitivity

Weight quantization maps a tensor to 8-bit integers with a scale per output channel. The sensitivity of a layer is measured without labels: its weights alone are replaced by their INT8 quantize–dequantize round trip, and the signal-to-quantization-noise ratio at the output logits is computed,

$$\text{SQNR}_\ell = 10 \log_{10} \frac{\|y\|^2}{\|y - \hat{y}_\ell\|^2}\ \text{dB}, \tag{6.2}$$

where $\hat{y}_\ell$ are the logits with layer ℓ perturbed. For L layers this takes $L + 1$ forward passes. A low SQNR marks a layer that quantization hurts. The same ranking supplies the layers kept in FP32 and the layers that receive a smaller pruning ratio, which gives a non-uniform allocation without the reinforcement-learning search of [3].

6.4.5 Budget Search

A budget is a string such as `latency<0.45ms,accuracy_drop<8%`. `Budget.parse` turns it into limits on latency, accuracy drop, minimum accuracy and size. The search tries the cheapest rung first and moves on only if the budget is unmet (Fig. 6.5); borderline passes are re-measured at full fidelity before being trusted, and prune trials that resolve

to an already measured architecture are skipped.

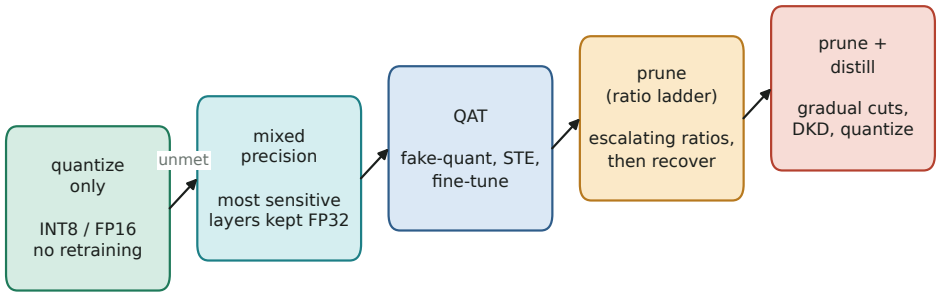


Figure 6.5: Escalation ladder of the budget search. Colours distinguish the rungs; the cheapest is on the left.

6.5 Data Flow Diagrams

6.5.1 DFD Level 0

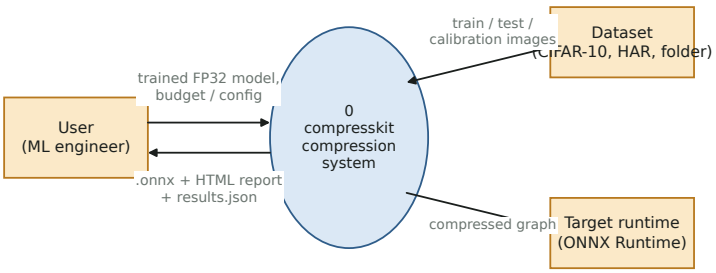


Figure 6.6: Data flow diagram, level 0: the toolkit as one process between the user, the dataset and the target runtime.

6.5.2 DFD Level 1

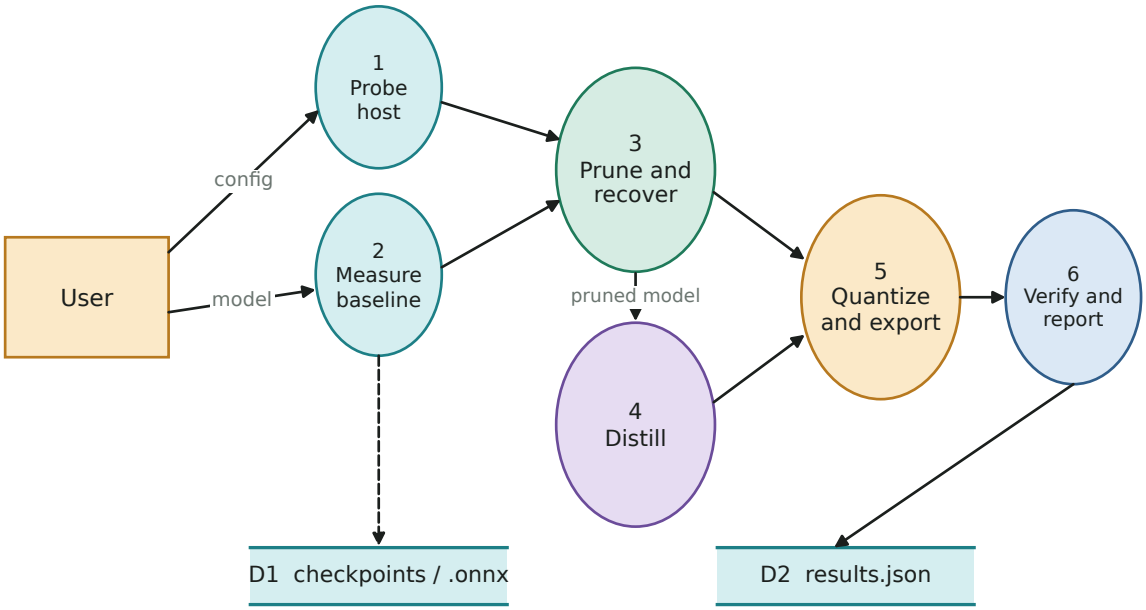


Figure 6.7: Data flow diagram, level 1: the six processes of one compression run.

6.5.3 DFD Level 2

Level 2 expands process 3 (prune and recover).

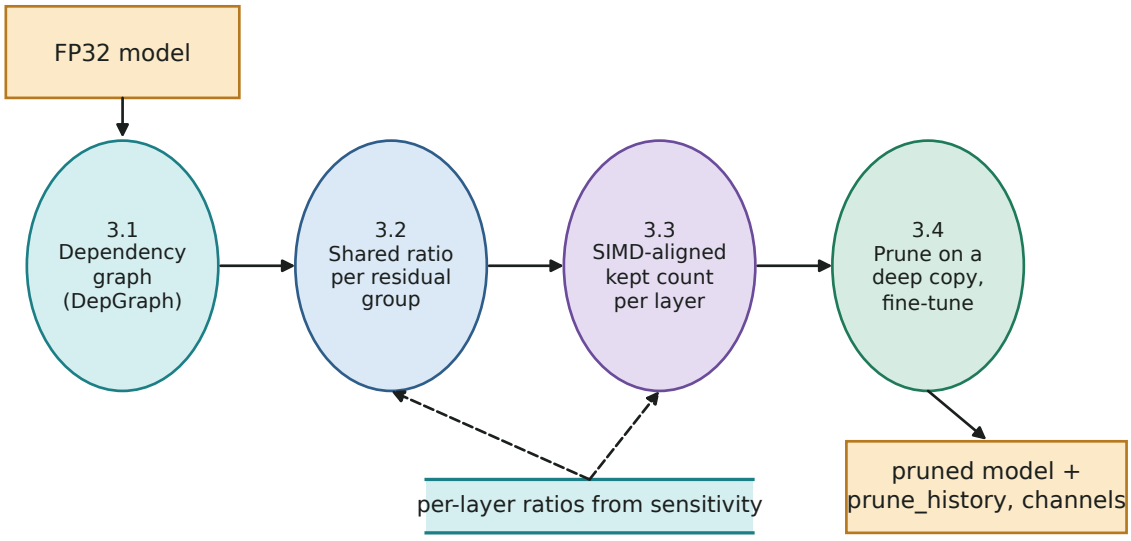


Figure 6.8: Data flow diagram, level 2: the pruning process.

6.6 UML Diagrams

6.6.1 Use Case Diagram

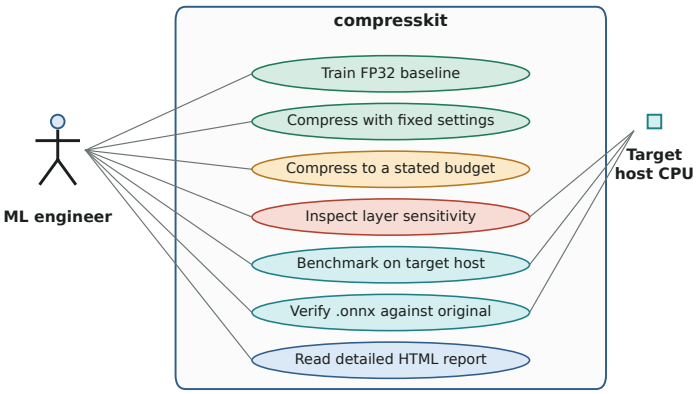


Figure 6.9: Use case diagram. The target host CPU is the secondary actor because latency and INT8 support are properties of that machine.

6.6.2 Sequence Diagram

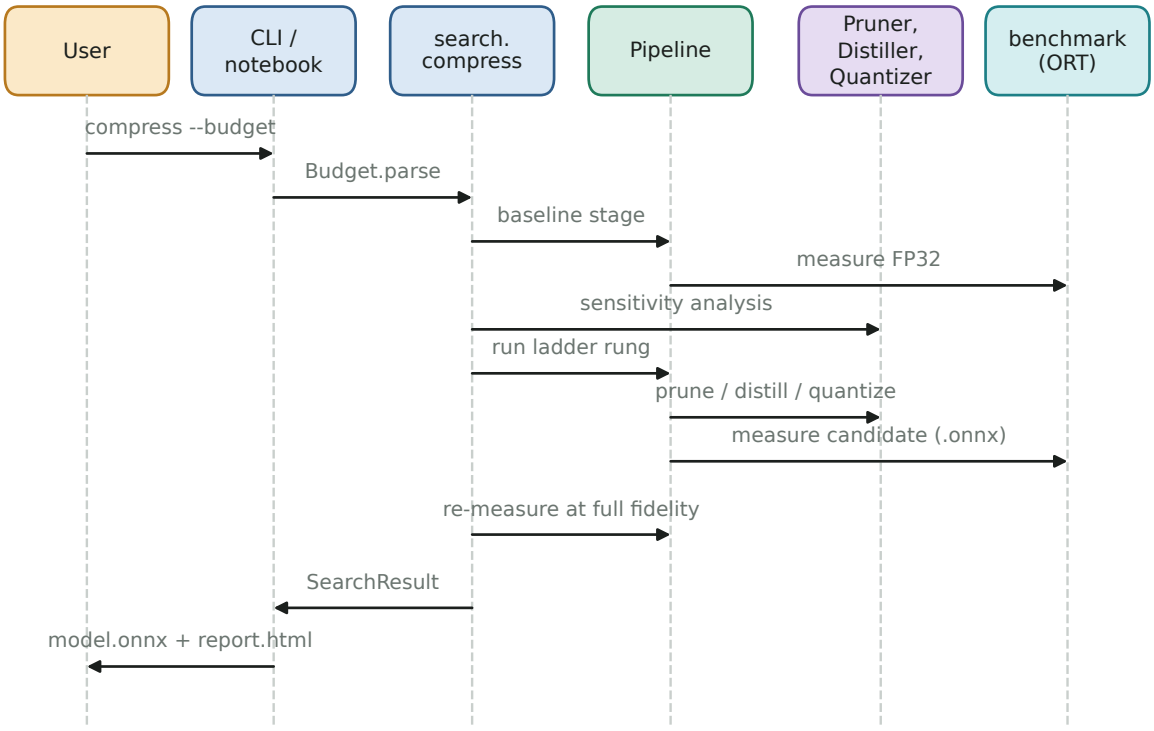


Figure 6.10: Sequence of a budget-driven `compress` run.

6.6.3 Class Diagram

The classes below are the main data carriers; each is a Python dataclass or a small class in the named module, and the field names are those in the source.

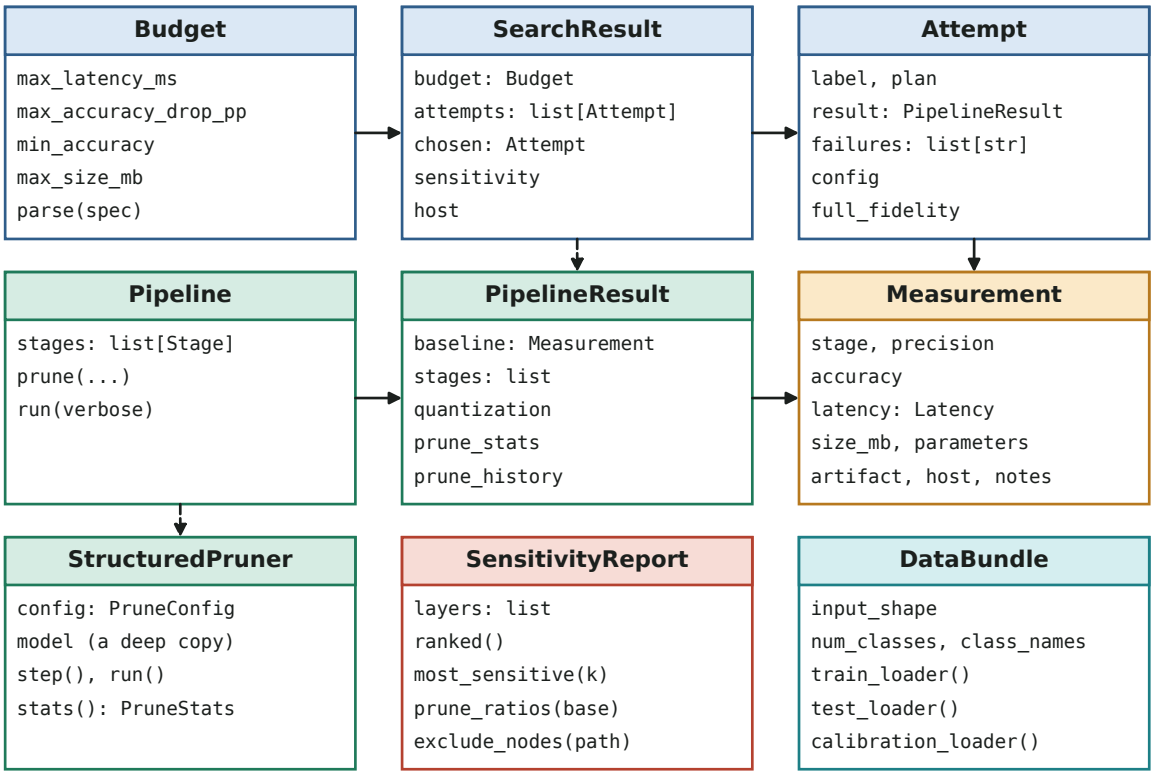


Figure 6.11: Class diagram of the principal types.

7. System Implementation

7.1 Package Layout

The package has 26 Python modules and 5,054 lines of source. The split into subpackages follows the stages, so a user can import the pruner without pulling in the quantizer. Project-specific code (the example networks and dataset loaders) lives in `zoo/` and is not part of the core API.

The largest modules are `search.py` (638 lines), `pruning/prune.py` (530), `report.py` (435), `pipeline.py` (420), `core/benchmark.py` (352) and `quantization/quantize.py` (338); the CLI is 329 lines and the example models, datasets and training code in `zoo/` about 700.

7.2 Command-Line Interface

The same flow is available from the command line. A complete run on CIFAR-10 is two commands:

```
compresskit train --arch resnet20
compresskit pipeline --model checkpoints/resnet20_fp32.pt \
    --prune 0.5 --steps 3 --fine-tune-epochs 2 --loss dkd \
    --precision auto --report report.html
```

To compress to a requirement instead of a fixed recipe:

```
compresskit compress --model checkpoints/resnet20_fp32.pt \
    --budget "latency<0.45ms,accuracy_drop<8%" --report report.html
```

Deployment latency on the target is re-measured with `compresskit bench model.onnx`, and the compressed file is compared with the original on real test images by the verification script `verify_onnx.py`.

7.3 Selected Code

The per-layer alignment rule of Algorithm 2, as implemented:

```
def _aligned_ratio(width, ratio, align):
    layer_align = align if align and align > 1 else 1
    while layer_align > 1 and width // layer_align < 2:
        layer_align //= 2
    if layer_align <= 1:
        kept = max(1, min(width, round(width * (1 - ratio))))
        return 1 - kept / width, layer_align
    desired_kept = width * (1 - ratio)
    max_blocks = width // layer_align
    n_blocks = min(max_blocks, max(1, round(desired_kept / layer_align)))
    kept = n_blocks * layer_align
    return 1 - kept / width, layer_align
```

The decoupled KD loss, including the numerical guard:

```
def binary(prob):
```

```
    target_p = prob[target_mask].unsqueeze(1).clamp(min=1e-7, max=1.0 - 1e-7)
    return torch.cat([target_p, 1.0 - target_p], dim=1)

tckd = F.kl_div(binary(student_prob).log(), binary(teacher_prob),
                reduction="batchmean") * (temperature ** 2)
masked_student = student_logits / temperature - 1000.0 * target_mask
masked_teacher = teacher_logits / temperature - 1000.0 * target_mask
# nckd: KL between the renormalised non-target distributions, scaled by T**2
return alpha * tckd + beta * nckd
```

7.4 One-Notebook Workflow

The project fixed one working environment. Code lives on GitHub; one Kaggle notebook (`kaggle/compresskit_end_to_end.ipynb`) does setup, data, training, compression, ONNX evaluation and report generation under *Run All*, driven by one configuration cell (model, mode manual or budget, prune ratio, steps, fine-tune epochs, loss, precision). Models are not copied between machines while the work runs; the output folder is downloaded once at the end and the final latency check runs on the local machine. Because the Kaggle CPU has no fast INT8, the notebook forces INT8 when asked and the report states that its latency is not the target's.

7.5 A Defect Found by Verification

The first end-to-end ResNet-20 run printed 88% accuracy in its log, but loading the exported `.onnx` and testing it on real images gave 47%. The cause was in the pipeline: the pruner operates on a deep copy of the model, and the final fine-tune step was applied to the original, unpruned model, while the exported graph came from the pruner's copy. The fix is one line, fine-tuning `pruner.model` instead of the caller's variable, and a regression test (`test_every_fine_tune_step_acts_on_the_pruned_model`) was written first, shown to fail on the old code and to pass after the change. This is why accuracy is measured on the exported file and not on the in-memory model.

8. Results and Testing

All numbers in this chapter were produced by the toolkit. Where a result is not from a full, final run, it is labelled.

8.1 Experimental Setup

Three CIFAR-10 networks were compressed with a requested channel ratio of 0.5, DKD fine-tuning, and INT8 quantization last. Table 8.1 states the schedule and where each was run.

Model	Schedule	Run on	Status
ResNet-20	3 steps × 2 DKD epochs	Apple M5 (INT8 fast)	Measured
ResNet-56	2 steps × 1 DKD epoch	Apple M5	Short smoke run, not the full schedule
VGG-16-BN	5 steps × 5 DKD epochs	Kaggle Xeon CPU (T4 for training)	Figures transcribed from the printed Kaggle log; latency is Kaggle-CPU latency

Table 8.1: Experimental runs and their status.

8.2 Compression Results

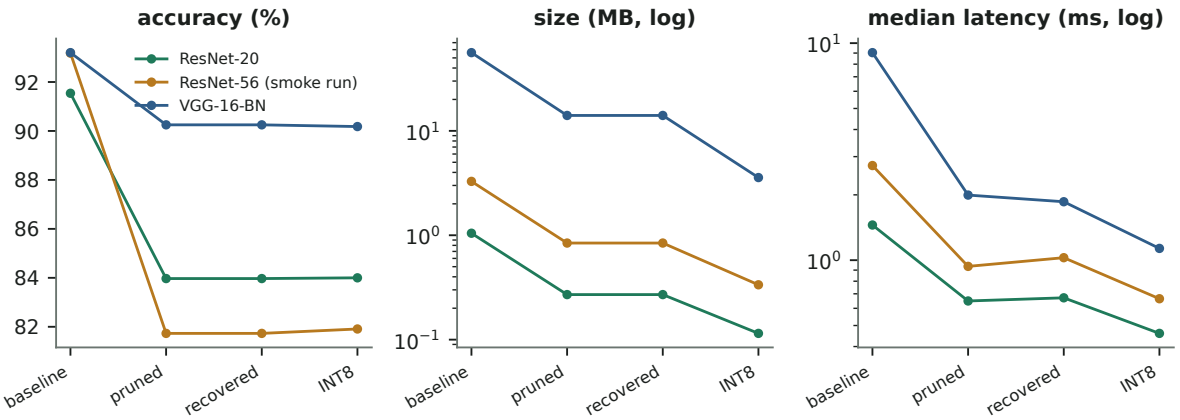


Figure 8.1: Accuracy, size and median latency through the four stages for the three recorded runs (size and latency on log axes).

Three observations follow. First, quantization alone moved accuracy by at most 0.18 points in these runs (ResNet-20 83.97 to 84.00, ResNet-56 81.73 to 81.91, VGG-16-BN

Model	Stage	Acc. (%)	Size (MB)	Latency (ms)	Parameters
ResNet-20	FP32 baseline	91.54	1.046	1.452	272,474
	pruned	83.97	0.270	0.649	68,786
	pruned and recovered	83.97	0.270	0.671	68,786
	INT8	84.00	0.115	0.460	68,786
ResNet-56	FP32 baseline	93.18	3.281	2.729	855,770
	pruned	81.73	0.842	0.936	215,282
	pruned and recovered	81.73	0.842	1.027	215,282
	INT8	81.91	0.335	0.664	215,282
VGG-16-BN	FP32 baseline	93.20	56.16	9.042	14,724,042
	pruned	90.25	14.05	1.995	3,684,842
	pruned and recovered	90.25	14.05	1.859	3,684,842
	INT8	90.18	3.57	1.133	3,684,842

Table 8.2: Per-stage measurements. ResNet-20 and ResNet-56 on Apple M5; ResNet-56 is the short smoke schedule; VGG-16-BN on a Kaggle CPU, transcribed from the run log.

Model	Size red.	Speed-up	Acc. cost (pp)	MACs (M)
ResNet-20	9.11×	3.16×	7.54	41.2 to 10.5
ResNet-56 (smoke)	9.79×	4.11×	11.27	126.8 to 32.1
VGG-16-BN (Kaggle CPU)	15.72×	7.98×	3.02	314.2 to 79.2

Table 8.3: Compression ratios, each beside the accuracy it cost. The VGG-16-BN speed-up is on a CPU without fast INT8 and is not a deployment figure.

90.25 to 90.18), so almost all of the accuracy cost belongs to pruning. Second, the pruned FP32 model is already about twice as fast as the baseline on ResNet-20 (1.452 to 0.649 ms); INT8 on a host with fused kernels adds a further drop to 0.460 ms. Third, the recovery stage by itself changed nothing visible once fine-tuning had been done, because the fine-tune had already re-estimated the statistics; its effect is isolated below.

8.3 Channels Removed

Fig. 8.2 shows the effect of the pruner on ResNet-20. Every layer is halved, with 16, 32 and 64 channels going to 8, 16 and 32. The classifier is untouched. Parameters fall from 272,474 to 68,786, a 74.8% reduction, because removing half the channels of a convolution removes about three quarters of its weights.

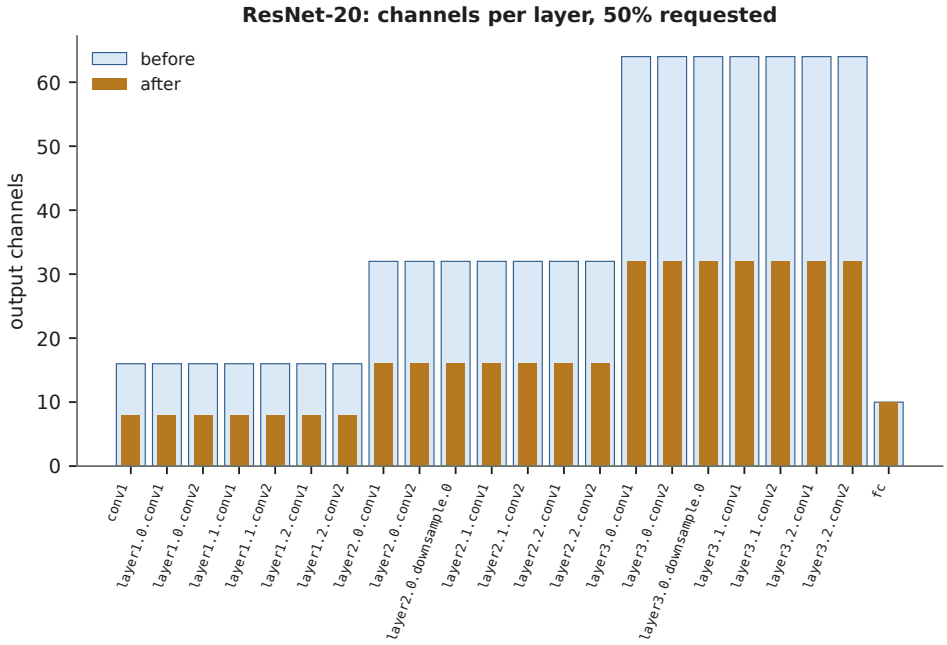


Figure 8.2: Output channels per layer of ResNet-20 before and after pruning.

8.4 Gradual Pruning

Fig. 8.3 shows the accuracy after each cut and after each fine-tune for ResNet-20 and VGG-16-BN. Each cut drops accuracy and each fine-tune recovers most of it. On ResNet-20 the cuts reach 69.28%, 63.36% and 49.82%, recovered to 87.63%, 86.26% and 83.97%. On VGG-16-BN the cuts are gentler (91.34% down to 86.66%) and recover to 91.77% down to 90.25%; the over-parameterised network loses less per cut than the compact ResNet-20.

8.4.1 Gradual against one-shot: a negative result

At an equal total of six fine-tuning epochs, three steps of two epochs ended at 83.97%, whereas one cut of the full 0.5 followed by six epochs ended at 85.03% (Fig. 8.4). One run each was made, so the 1.06-point difference is not established, but the data do not show gradual pruning to be better on this model and the report does not claim that it is. Gradual pruning is kept as an option because the budget search uses it in its recovery rung.

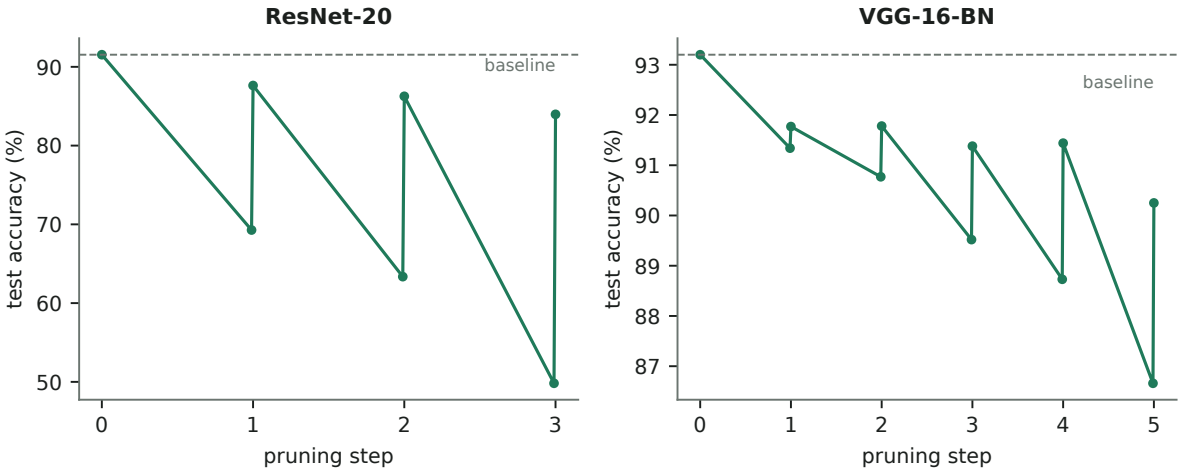


Figure 8.3: Accuracy after each cut and after each fine-tune. ResNet-20 on Apple M5; VGG-16-BN transcribed from the Kaggle run.

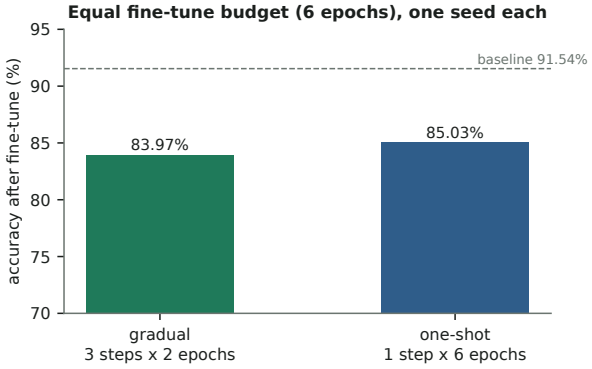


Figure 8.4: Gradual and one-shot pruning at the same fine-tune budget on ResNet-20, one seed each.

8.5 Batch-Norm Re-estimation

After a one-shot 0.5 cut with no fine-tuning, ResNet-20 falls from 91.54% to 10.87%, close to chance for ten classes. Re-estimating the batch-norm statistics from training batches raises it to 17.75% (Fig. 8.5). The left panel shows why the correction is needed: the running variance of the surviving channels of `layer1.0.bn1` after recalibration differs from the stale values copied from the old network. The correction is real but small; it does not replace fine-tuning, which is why distillation follows it in the pipeline.

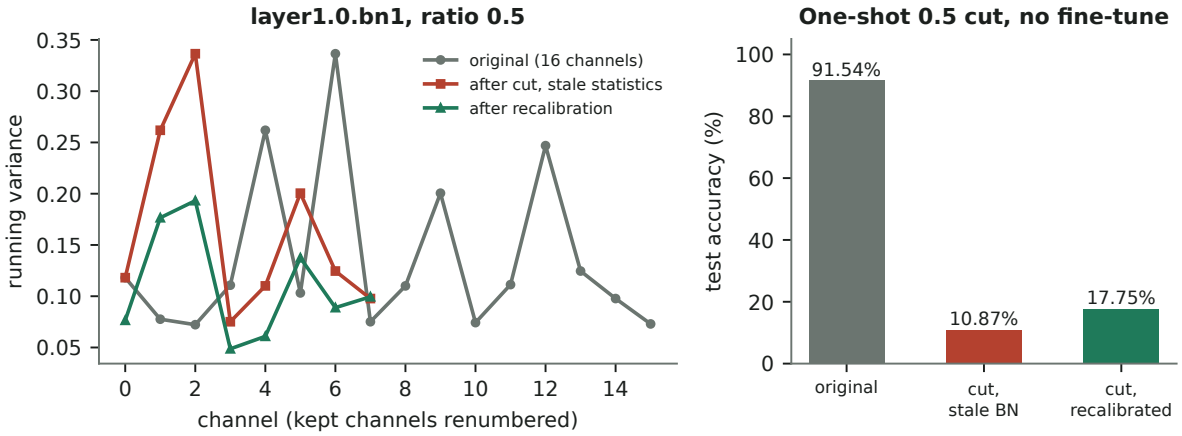


Figure 8.5: Running variance of one batch-norm layer before and after the cut (left) and the accuracy effect (right).

8.6 Distillation Signal

Fig. 8.6 shows the teacher and student softmax outputs on one real test image of a cat, for the one-shot-pruned student that has not yet been fine-tuned. At temperature 1 the teacher gives 0.9991 to the true class and almost nothing elsewhere, so the other classes carry little information. At temperature 4 the teacher gives 0.7474 to cat and a visible ranking over the other nine classes; the student gives cat 0.0627 and favours horse. The NCKD term trains the student to reproduce the teacher's ranking over the non-target classes, and TCKD trains it on the target probability.

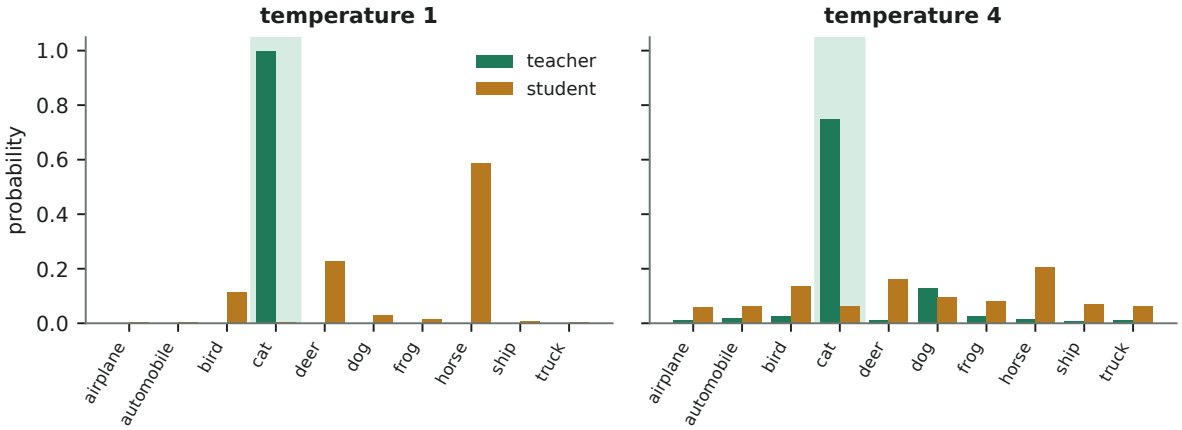


Figure 8.6: Teacher and student softmax on one test image at temperature 1 (left) and 4 (right).

8.7 Quantization

Fig. 8.7 shows one convolution of ResNet-20 (`layer2.0.conv1`, $32 \times 16 \times 3 \times 3$) before and after the INT8 round trip. The two histograms are nearly the same: the weight mean-squared error is 2.1×10^{-7} and 248 of the 256 integer levels are used. The per-channel scales (right) differ across output channels by a factor of nearly two, which is why per-channel quantization is used rather than one scale for the whole tensor. In every run, all compute operators were quantized (22 of 22 for ResNet-20: 21 convolutions and one Gemm).

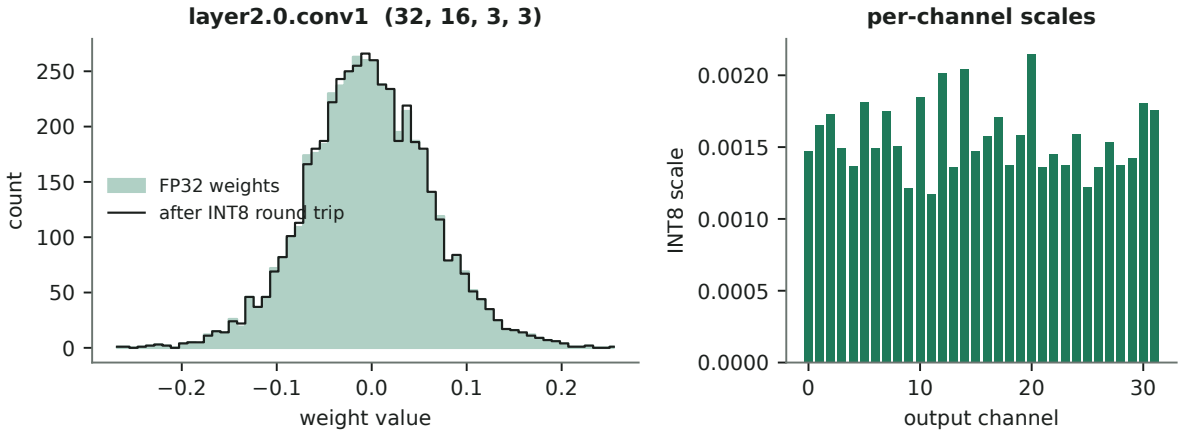


Figure 8.7: Weight histogram of one layer before and after INT8 (left) and the INT8 scale of each output channel (right).

8.8 Layer Sensitivity

Across the 22 layers of ResNet-20, quantization of a single layer produced an SQNR at the logits between 45.64 dB (`layer1.2.conv1`) and 62.34 dB (`layer3.2.conv2`), measured on four batches (Fig. 8.8). The two most sensitive layers, `layer1.2.conv1` and `layer3.1.conv1`, are the ones the toolkit protects at a ratio of 0.5. The spread is 17 dB, so layers are not equally fragile and a single ratio for all layers leaves accuracy on the table.

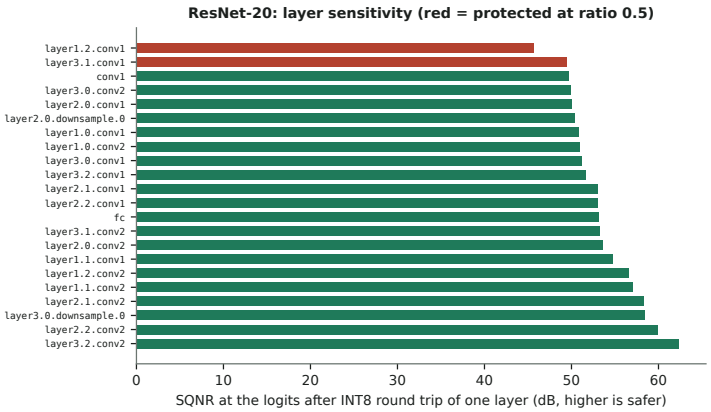


Figure 8.8: Per-layer SQNR of ResNet-20 after INT8 round trip of one layer at a time. Lower means more sensitive.

8.9 Budget Search

Three budgets were given to the search on ResNet-20 with shortened recovery schedules (Fig. 8.9, Table 8.4).

Budget	Configuration chosen	Met	Accuracy (%)	Latency (ms)
accuracy drop < 1 pp, size < 0.4 MB	quantize only	yes	91.54	0.878
latency < 0.45 ms, drop $<$ 8 pp	prune 0.788, distill, quantize	yes	84.96	0.361
latency < 0.02 ms, drop $<$ 0.5 pp	best trade-off (same configuration)	no	84.28	0.355

Table 8.4: Outcomes of the budget search. The third budget is infeasible by design.

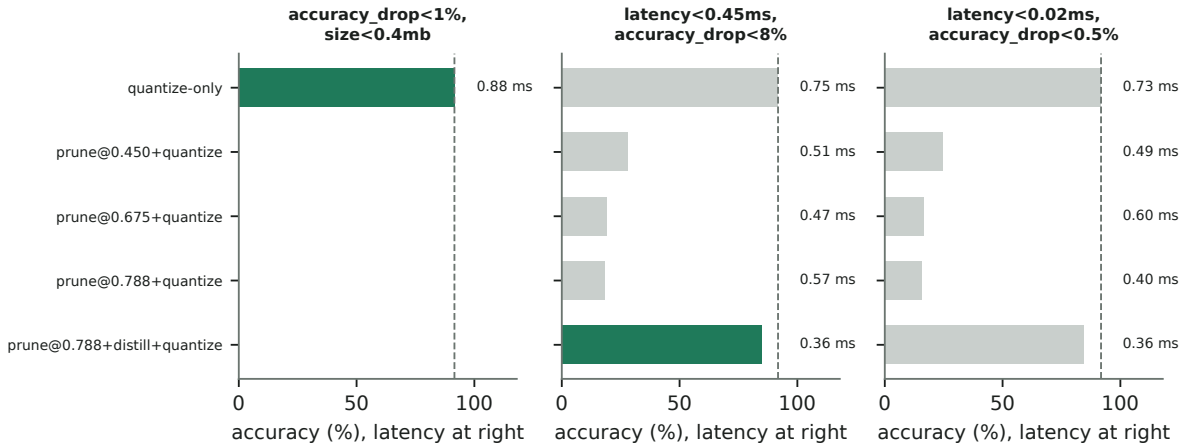


Figure 8.9: Every attempt made under each budget. Green bars met the budget; the dashed line is the baseline accuracy.

The second budget shows why the ladder exists. Quantization alone gave 0.747 ms, above the 0.45 ms limit. Pruning without recovery reached the latency range but collapsed accuracy to 18–28%; only the rung that adds gradual fine-tuning with DKD met both limits. The third budget could not be met, and the search returned the most accurate candidate that came closest and said that the budget was missed. The measured latencies of the same file differ between searches (0.747 and 0.734 ms for quantize-only) because search-time measurements are short; borderline results are re-measured at full fidelity before being trusted.

8.10 Verification of the Compressed ONNX File

`experiments/verify_onnx.py` runs the original PyTorch model and the compressed ONNX file on the same real CIFAR-10 test images. On 1,000 test images, the original ResNet-20 reached 92.30% and the compressed file 83.20%; the two models gave the same prediction on 86.1% of images. The file therefore works and is close to the original in the sense that matters for a deployed classifier, but it is not identical, and the gap is of the size the pipeline reported (7.5 points).

8.11 Testing

The toolkit has 162 automated tests in five files (core 104, pipeline 31, search 21, workflow 4, dataset registry 2), run by default, and one slow end-to-end test (train, gradual prune, INT8 quantize, report) that is deselected by default and takes about three and a half minutes. Tests cover the host probe and its feature parsing, the alignment rule, dependency-group resolution, pruning edge cases found during review, the KD and DKD losses including the NaN guard, budget parsing, the search ladder, ONNX export and the dataset registry. The stale-model defect of Section 7.5 is covered by a regression test.

8.12 Limitations

- The ResNet-56 result is a two-step, one-epoch smoke run; the full five-step schedule was prepared as a notebook and not yet recorded. Its 11.27-point accuracy cost should not be read as the method's ability on this model.
- The VGG-16-BN figures were transcribed from the Kaggle log and not loaded from the run's `results.json`; its latency is that of a Kaggle CPU without fast INT8.
- Each experiment was run once; no confidence intervals are given, and differences of about one point (such as gradual against one-shot) are within what a different seed could change.
- The ResNet-20 INT8 accuracy cost of 7.54 points is large. Longer fine-tuning or QAT may reduce it, but that was not measured here.
- Only residual and plain stacked CNNs on CIFAR-10 were tested. MobileNet-style depthwise networks, larger-resolution data, and the HAR path on a GPU run were not.

9. Conclusion and Future Scope

9.1 Summary

CompressKit composes structured pruning, batch-norm re-estimation, distillation and quantization in a fixed order, measures everything on the exported ONNX file, and searches for the cheapest combination that meets a stated budget. Four mechanisms distinguish it from running the techniques separately: a host probe that gates INT8, a per-layer alignment rule, shared ratios across residual groups, and a label-free sensitivity ranking. On CIFAR-10, ResNet-20 became 9.1 times smaller and 3.2 times faster on an Apple M5 at a 7.54-point accuracy cost, and a VGG-16-BN run shrank 15.7 times at a 3.02-point cost.

9.2 Conclusions

- In these runs the accuracy cost came from pruning, not from INT8 quantization, which changed accuracy by at most 0.18 points.
- Fine-tuning with distillation after each cut recovers most of the accuracy lost by that cut, but at an equal budget gradual pruning did not beat one-shot on ResNet-20.
- Batch-norm re-estimation helps (10.87% to 17.75% after a 50% cut) but is not a substitute for fine-tuning.
- The budget search is useful because the cheap rungs often do not suffice: pruning without recovery met a latency limit and destroyed accuracy, and only the distillation rung met both.

9.3 Future Scope

- Run the full ResNet-56 schedule and the HAR ResNet-18 path on Kaggle, load the recorded `results.json` files, and repeat each experiment with several seeds to report variance.
- Add support and tests for depthwise-separable networks such as MobileNet.
- Evaluate QAT and longer fine-tuning on the 7.54-point ResNet-20 gap.
- Add detection and segmentation models, and non-CPU runtimes with their own latency measurement.

Bibliography

[1] W. Chae and K. Seo, "Pipeline of Pruning, Knowledge Distillation, and Quantization for Model Compression," *J. Electrical Engineering & Technology*, vol. 21, pp. 2099–2105, 2026, doi: 10.1007/s42835-025-02541-7.

[2] Z. Wang, L. Shi, Z. Mei, X. Zhao, Z. Wang, and J. Li, "Iterative knowledge distillation and pruning for model compression in unsupervised domain adaptation," *Pattern Recognition*, vol. 164, art. 111512, 2025.

[3] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "AMC: AutoML for Model Compression and Acceleration on Mobile Devices," in *Proc. ECCV*, 2018.

[4] M. Zhou, R. Mao, S. Sripada, and X. Li, "AEMC: Aggregator-efficient model compression for streamlining LLMs," *Neurocomputing*, vol. 672, art. 132779, 2026.

[5] W. Chen *et al.*, "Quantization of Deep Neural Networks for Accurate Edge Computing," *ACM J. Emerging Technologies in Computing Systems*, vol. 17, no. 4, art. 54, 2021.

[6] R. Abdel-Salam, A. H. Abdel-Gawad, and A. G. Wassal, "VLCQ: Post-training quantization for deep neural networks using variable length coding," *Future Generation Computer Systems*, vol. 166, art. 107654, 2025.

[7] N. A. Khan and A. M. S. Rafat, "Optimizing Deep Learning Models for Resource-Constrained Environments With Cluster-Quantized Knowledge Distillation," *Engineering Reports*, vol. 7, art. e70187, 2025.

[8] X. Fang, L. Chen, H. Yin, X. Chen, and W. Wang, "Communication Efficient Federated Learning With Quantization-Aware Training Design," *IEEE Trans. Machine Learning in Communications and Networking*, vol. 4, pp. 45–59, 2026.

[9] Y. Chen, C. Li, L. Gong, X. Wen, Y. Zhang, and W. Shi, "A deep neural network compression algorithm based on knowledge transfer for edge devices," *Computer Communications*, vol. 163, pp. 186–194, 2020.

[10] L. Xu, J. Ren, Z. Huang, W. Zheng, and Y. Chen, "Improving Knowledge Distillation via Head and Tail Categories," *IEEE Trans. Circuits and Systems for Video Technology*, vol. 34, no. 5, pp. 3465–3478, 2024.

[11] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards Any Structural Pruning," in *Proc. IEEE/CVF CVPR*, 2023, pp. 16091–16101.

[12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.

[13] B. Zhao, Q. Cui, R. Song, Y. Qiu, and J. Liang, "Decoupled Knowledge Distillation," in *Proc. IEEE/CVF CVPR*, 2022, pp. 11953–11962.

[14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[15] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. ICLR*, 2015.

[16] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Tech. Rep., University of Toronto, 2009.

[17] B. Jacob *et al.*, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.

[18] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," arXiv:1308.3432, 2013.